

Documentation du Projet DND Game

1. Introduction

Le projet "DND Game" est un jeu de rôle de type dungeon crawler développé en Python avec l'interface graphique ezTK. Le jeu propose une expérience de type RPG où le joueur peut naviguer à travers différents niveaux (paliers), combattre des ennemis variés, gagner de l'expérience et progresser jusqu'à affronter un boss final.

Ce document présente les choix algorithmiques, la structure du code, les fonctionnalités principales et les limitations actuelles du projet.

2. Structure du projet

Le projet est organisé en plusieurs modules principaux :

- **DNDGame** : Classe principale qui gère la logique du jeu
- **GameInterface** : Gère l'affichage et l'interface utilisateur
- **GameActions** : Contrôle les actions du joueur et des ennemis
- **Player** : Définit les caractéristiques du joueur
- **Enemy** : Contient la définition des différents types d'ennemis
- **Race et Classe** : Modules fournissant des modificateurs pour le joueur

2.1 Diagramme des classes

```
DNDGame
├── GameInterface
├── GameActions
├── Player
│   ├── Race (Human, etc.)
│   └── Classe (Warrior, etc.)
└── Enemy
    ├── Cutiie
    ├── Goblin
    ├── Orc
    ├── Troll
    └── Dragon
```

└─ Vampire
└─ Boss

3. Fonctionnalités principales

3.1 Système de progression par paliers

Le jeu est structuré en paliers (niveaux) de difficulté croissante. À chaque palier :

- Le nombre et la puissance des ennemis augmentent
- L'apparence de l'environnement change (couleurs différentes)
- De nouveaux types d'ennemis apparaissent

Ce système permet une progression graduelle de la difficulté tout en maintenant un défi constant pour le joueur.

3.2 Système de combat tour par tour

Le combat est géré par un système de tour par tour :

1. Le joueur dispose de 2 actions par tour
2. Les actions possibles sont : se déplacer et attaquer
3. Après avoir utilisé ses actions, c'est au tour des ennemis
4. Les ennemis peuvent se déplacer ou attaquer en fonction de leur proximité avec le joueur

```
def end_turn(self):
    """
        End the player's turn and switch to the enemy's turn.
    """
    if self.game_over:
        return

    if self.current_turn == "player":
        self.current_turn = "enemies"
        self.ui.status_label.config(text="Enemies' turn")
        self.player.actions = 2
        self.ui.root.after(500, self.act.enemy_action)
    else:
        # Update actions display
        self.ui.action_label.config(text=f"Actions left: {self.player.actions}")
        self.current_turn = "player"
        self.ui.status_label.config(text="Player's turn")
```

3.3 Système de personnage avec statistiques et progression

Le joueur possède :

- Une race et une classe qui influencent ses statistiques
- Un niveau calculé à partir de son expérience
- Des points de vie qui augmentent avec le niveau
- Des bonus de mouvement, d'attaque et de défense

```
def calculate_level(self):
    """Calcule le niveau en fonction de l'XP"""
    level = 1
    xp_threshold = 100

    while self.xp >= xp_threshold:
        level += 1
        xp_threshold += 100 * level
    return level

def gain_xp(self, amount):
    """Ajoute de l'XP au joueur et gère la montée de niveau"""
    old_level = self.level
    self.xp += amount

    new_level = self.calculate_level()
    if new_level > old_level:
        self.level = new_level
        self.xp_to_next_level = 100 * self.level
        for _ in range(new_level - old_level):
            self.health += 5
```

3.4 Système de sauvegarde des données joueur

Le jeu enregistre la progression du joueur dans un fichier JSON, permettant de reprendre une partie plus tard :

```
def set_bdd(self):
    """Set player data in database"""
    data = get_all_data()
    for player in data.get('players', []):
        if player['name'] == self.name:
            player['xp'] = self.xp
            player['palier'] = self.palier
            player['health'] = self.health
```

```

        player['level'] = self.level
        player['race'] = self.race_name
        player['classe'] = self.classe_name
        break

    with open('players_database.json', 'w') as f:
        json.dump(data, f, indent=4)
    return

```

4. Choix algorithmiques

4.1 Calcul des déplacements possibles

Pour déterminer les cases accessibles lors du déplacement du joueur, un algorithme de distance circulaire est utilisé plutôt qu'une distance de Manhattan ou de case à case. Cette approche permet des mouvements plus fluides et naturels :

```

def possible_coords(self, start_coord, move_distance):
    """Calculate possible coordinates within a movement distance"""
    possible_moves = []
    for dx in range(-move_distance, move_distance + 1):
        for dy in range(-move_distance, move_distance + 1):
            if dx**2 + dy**2 > move_distance**2: # Use circular distance
                continue
            new_x = start_coord[0] + dx
            new_y = start_coord[1] + dy
            if 1 <= new_x < 19 and 1 <= new_y < 39 or (new_x, new_y) ==
(10,0) or (new_x, new_y) == (9,0) or (new_x, new_y) == (10,39) or
(new_x, new_y) == (9,39):
                possible_moves.append((new_x, new_y))
    return possible_moves

```

4.2 Attaque du Boss

Pour l'attaque du boss final, un algorithme de tracé de ligne basé sur l'algorithme de Bresenham est utilisé pour calculer le chemin de l'attaque. Cette approche permet de créer des attaques directionnelles visuellement intéressantes :

```

def __grid_boss_next_attack(self, attack_type):
    if attack_type == "Fireball":
        # [...] Détermination initiale du type d'attaque

        # Cas d'une attaque diagonale
    else:

```

```

dx = 1 if player_x > boss_x else -1
dy = 1 if player_y > boss_y else -1

rise = abs(player_y - boss_y)
run = abs(player_x - boss_x)

is_more_horizontal = run > rise
x, y = boss_x + dx, boss_y + dy

# Use Bresenham's line algorithm (enhanced by claude AI)
error = 0
if is_more_horizontal:
    error_step = rise / run
    while 0 < x < MAX_X and 0 < y < MAX_Y:
        for offset in range(3):
            offset_y = y - 1 + offset
            if 0 < offset_y < MAX_Y:
                position.append((x, offset_y))
        error += error_step
        x += dx
        if error >= 0.5:
            y += dy
            error -= 1.0
    else:
        # [...] Traitement similaire pour les lignes plus verticales

```

4.3 Équilibrage des statistiques du joueur

Pour éviter les personnages trop puissants, un système de plafonnement des bonus est utilisé. Chaque statistique est calculée comme une somme de bonus de base, de niveau, de race et de classe, puis plafonnée à une valeur maximale :

```

def bonus_attack(self):
    """Calculate a balanced bonus for the player's attack."""
    base_bonus = random.randint(1, 6)
    level_bonus = int(self.level * 2)
    bonus_race = self.info_race.attack(base_bonus)
    bonus_classe = self.info_classe.attack()
    total_bonus = base_bonus + level_bonus + bonus_race + bonus_classe

    max_bonus = 20
    total_bonus = min(total_bonus, max_bonus)

    return total_bonus

```

5. Interface graphique

L'interface graphique est construite avec ezTK et comprend :

- Une grille de jeu pour visualiser le joueur et les ennemis
- Un panneau d'informations sur le joueur
- Un journal d'actions
- Des boutons pour les actions (déplacement, attaque)

Les éléments visuels sont mis à jour en temps réel pour refléter l'état du jeu, notamment :

- La position du joueur et des ennemis
- Les cases accessibles lors d'un déplacement ou d'une attaque
- Les attaques des ennemis (notamment les animations d'attaque du boss)

5.1 Animations

Un soin particulier a été porté aux animations pour enrichir l'expérience utilisateur :

```
def animate_attack_execute(self, cell_groups, attack_type):
    """
    Execute phase animation - attack visualization
    """
    attack_colors = {
        "Fireball": ["#FFDD00", "#FF9500", "#FF5500", "#FF0000"] # Yellow to
red
    }
    colors = attack_colors.get(attack_type, attack_colors["Fireball"])

    previous_groups = []
    for i, group in enumerate(cell_groups):
        color_index = min(i % len(colors), len(colors) - 1)
        for cell in group:
            row, col = cell
            if 0 <= row < 19 and 0 <= col < 39:
                self.cells[col][row].config(bg=colors[color_index])
                if (row, col) == self.player.coord:
                    self.cells[col][row].config(bg="blue")
    # [...] Gestion des effets de traînée
```

6. Limitations actuelles et perspectives d'amélioration

6.1 Limitations

- **Intelligence artificielle des ennemis** : Les ennemis suivent des schémas de comportement simples, sans véritable stratégie.
- **Variété des actions** : Le nombre d'actions disponibles pour le joueur est limité (déplacement et attaque).
- **Absence d'objets** : Aucun système d'objets ou d'équipement n'est implémenté.
- **Feedback visuel** : Certaines actions manquent de retour visuel clair.

6.2 Perspectives d'amélioration

1. **Système d'objets** : Ajouter des potions, équipements et trésors qui peuvent être collectés.
2. **Capacités spéciales** : Implémenter des compétences uniques pour chaque classe.
3. **IA améliorée** : Développer des comportements plus complexes pour les ennemis.
4. **Effets visuels** : Améliorer les animations et les effets visuels.
5. **Génération procédurale** : Créer des niveaux générés de façon aléatoire pour une rejouabilité accrue.

7. Bilan chronologique du développement

Le développement du jeu s'est déroulé en plusieurs phases :

1. **Conception initiale** : Définition des règles de base, de la structure des classes et de l'interface.
2. **Développement du moteur de jeu** : Mise en place de la logique de tour par tour et des déplacements.
3. **Développement de l'interface graphique** : Création de la grille et des éléments visuels.
4. **Création du système de combat** : Implémentation des attaques et des défenses.
5. **Système de progression** : Ajout des paliers, de l'expérience et des niveaux.
6. **Boss final** : Création d'un boss avec des attaques spéciales.
7. **Polissage** : Ajout d'animations, correction de bugs et équilibrage du jeu.

8. Conclusion

Le projet DND Game offre une expérience de jeu de rôle simplifiée mais complète, avec un système de progression, des combats stratégiques et une interface graphique fonctionnelle. Malgré certaines limitations, le jeu propose un gameplay satisfaisant et une structure de code modulaire qui faciliterait les extensions futures.

Ce projet a permis d'explorer différents aspects du développement de jeux, notamment la gestion des tours, l'interface utilisateur, les animations et l'équilibrage des mécaniques de jeu.